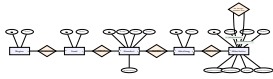


Aufgabe 1 Entity Relationship Modell

1.2 & 1.3 Region/Land/Standort/Abteilung/Mitarbeiter

Region(ID,Name) → 1:n → Land(ID,Name) → 1:n → Standort(ID,Straße,PLZ,Stadt,Bundesland) → 1:n → Abteilung(ID,Name) → 1:n → Mitarbeiter(ID,Vorname,Nachname,E-Mail,Telefonnummer,Einstelldatum,Gehalt,Provisionsatz), Mitarbeiter hat rekursive 1:n-Beziehung "ist Vorgesetzter von" auf sich selbst.

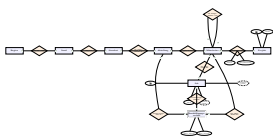
► **Lösung:** Chen-Notation: Rechteck=Entität, Raute=Beziehung, Ellipse=Attribut (unterstr.), str.=Schlüssel, gestrichelt=abgeleitet), Zahlen=Kardinalität.



1.4-1.6 Projekte, Jobs, Beschäftigungsverlauf

Projekt(ID,Name) steht in n:m-Beziehung "arbeitet mit" zu Mitarbeiter (Beziehungsattribute: Anteil, Projektleitung – n:m-Beziehungen dürfen eigene Attribute tragen). Job(ID,Titel,Mindestgehalt*,Maximalgehalt* [*abgeleitet]) steht in 1:n-Beziehung "ist angestellt als" zu Mitarbeiter. Beschäftigungsverlauf (Start_Datum,End_Datum) ist eigener schwacher Entitätstyp mit je einer 1:n-Beziehung zu Mitarbeiter/Abteilung/Job (da keine Paarkombination die Einträge eindeutig identifiziert; PK später: Mitarbeiter-ID+Start_Datum).

► **Lösung:** Vollständiges ER-Modell (Basis für Aufgabe 2). Attribute von Region/-Land/Standort/Abteilung/Mitarbeiter siehe Diagramm oben.



Aufgabe 2 relationales Modell

Im Rahmen dieser Übung soll aus dem Entity-Relationship-Modell (siehe Lösung zu Aufgabe 1.6) ein relationales Modell erstellt werden.

2.1 Relationen der Entitätstypen

Überführen Sie alle Entitätstypen aus dem Entity-Relationship-Modell in das relationale Modell.

► **Lösung:** Jeder Entitätstyp wird zunächst 1:1 in eine Relation überführt; die Schlüsselattribute des Entitätstyps werden zum Primärschlüssel der Relation:

- region(region_id, region_name)
- country(country_id, country_name)
- location(location_id, street_address, postal_code, city, state_province)
- department(department_id, department_name)
- employee(employee_id, first_name, last_name, email, phone_number, hire_date, salary, commission_pct)
- job(job_id, job_title, min_salary, max_salary)
- project(project_id, project_name)

Die abgeleiteten Attribute min_salary/max_salary werden trotz ihrer Ableitbarkeit als gewöhnliche (materialisierte) Spalten geführt, da SQL keine automatisch berechneten, immer aktuellen Attribute ohne Trigger/View kennt.

2.2 Relationen der Beziehungstypen

Überführen Sie alle Beziehungstypen des Entity-Relationship-Modells in das relationale Modell. Achten Sie insbesondere auf die korrekte Definition der Primärschlüssel.

► **Lösung:** Grundregel: 1:n-Beziehungen benötigen keine eigene Relation – der Fremdschlüssel wird stattdessen als zusätzliche Spalte in die Relation auf der "n"-Seite aufgenommen. Nur n:m-Beziehungen (und mehrstellige/assoziative Beziehungen) benötigen eine eigene Verbindungsrelation.

- ist Teil von (1:n): country.region_id → region.region_id
- liegt in (1:n): location.country_id → country.country_id
- ist angesiedelt in (1:n): department.location_id → location.location_id
- gehört zu (1:n): employee.department_id → department.department_id
- ist angestellt als (1:n): employee.job_id → job.job_id
- ist Vorgesetzter von (1:n, rekursiv): employee.manager_id → employee.employee_id
- arbeitet mit (n:m) ⇒ eigene Relation): employee2project(employee_id, project_id,

assoc_pct, is_project_lead) mit employee_id → employee.employee_id und project_id → project.project_id. Der Primärschlüssel besteht aus beiden Fremdschlüsseln, da genau diese Kombination die Beziehung eindeutig identifiziert.

- Beschäftigungsverlauf (drei 1:n-Beziehungen zu einem assoziativen Entitätstyp ⇒ eigene Relation): job_history(employee_id, start_date, end_date, job_id, department_id) mit employee_id → employee.employee_id, job_id → job.job_id, department_id → department.department_id. Als Primärschlüssel wird – wie in der Aufgabenstellung vorgegeben – (employee_id, start_date) verwendet (ein Mitarbeiter kann zu einem gegebenen Startdatum immer nur einen Beschäftigungsabschnitt beginnen).

2.3 Vereinfachung des relationalen Modells

Vereinfachen Sie das entstandene relationale Modell durch geeignete Zusammenfassung der Relationen.

► **Lösung:** Da sämtliche 1:n-Beziehungen bereits in Schritt 2.2 direkt als Fremdschlüssel-Spalte in die referenzierende Relation integriert wurden (statt eigene Beziehungsrelationen anzulegen), ist keine weitere Zusammenfassung mehr nötig oder möglich: Eine zusätzliche Verschmelzung wäre nur bei 1:1-Beziehungen sinnvoll (hier nicht vorhanden) oder wenn fälschlicherweise auch für 1:n-Beziehungen eigene Relationen angelegt worden wären. Das vereinfachte relationale Modell besteht somit aus genau 9 Relationen: region, country, location, department, job, employee, project, employee2project, job_history.

2.4 Datentypen

Erweitern Sie die Relationen des vereinfachten relationalen Modells um passende Datentypen.

► **Lösung:** Die folgende Abbildung zeigt das vollständige relationale Modell (= HR-Schema) inklusive Datentypen, Primär-(unterstrichen) und Fremdschlüsseln (Pfeile). Es ist identisch mit dem in Aufgabe 3.6/4 vorausgesetzten Schema und bildet die Grundlage für alle folgenden Übungen.



Legende: Pfeile zeigen jeweils vom Fremdschlüssel auf den referenzierten Primärschlüssel. Der Selbstbezug employee.manager_id → employee.employee_id ist als kleine Schleife an der Employee-Box angedeutet. Die vollständigen CREATE TABLE-Anweisungen inkl. aller Fremdschlüssel folgen in der Lösung zu Aufgabe 4.

Aufgabe 3 Normalisierung

3.1 Normalformen

Gegeben sei das folgende relationale Modell zur Verwaltung der offenen Rechnungen eines Onlineshops inkl. Beispieldaten:

RNr	KDNr	Name	Wohnort
1	1	Müller	München
2	1	Müller	München
3	2	Huber	Nürnberg
4	2	Huber	Nürnberg
5	3	Meier	Augsburg
6	4	Meier	München

3.1.1 2. Normalform

Warum können nur Relationen mit zusammengesetzten Schlüsseln die 2. NF verletzen?

► **Lösung:** 2. NF verletzt = ein Nicht-Schlüsselattribut hängt nur von einer echten Teilmenge des Schlüssels ab (partielle Abhängigkeit). Ein einwertiger Schlüssel hat keine nichttriviale echte Teilmenge ⇒ partielle Abhängigkeit strukturell unmöglich. Erst bei zusammengesetztem Schlüssel kann ein Attribut von einem Teil davon abhängen.

3.2 Funktionale Abhängigkeiten

► **Lösung:** Schlüssel: RNr. FDs: RNr→KDNr,Pos,Datum,Betrag; KDNr→Name,Wohnort. Transitiv: RNr→Name,Wohnort.

3.3 3. Normalform

► **Lösung:** Verstoß: Name/Wohnort hängen nur transitiv via KDNr vom Schlüssel RNr ab. Anomalien: Änderung (Umgang) ⇒ mehrere Zeilen ändern), Einfügen (Kunde ohne Rechnung nicht speicherbar), Löschen (letzte Rechnung löschen ⇒ Kundendaten weg). 3. NF: Kunde (KDNr,Name,Wohnort), Rechnung (RNr,KDNr,Pos,Datum,Betrag) mit KDNr→Kunde.KDNr.

3.4 Boyce-Codd Normalform

► **Lösung:** Beide Relationen: einziger Determinant ist jeweils der (einwertige) Schlüssel selbst ⇒ Zerlegung aus 3.3 ist bereits BCNF, keine weitere nötig.

3.5 Normalform Hochschul-Schema

► **Lösung:** Dozent/Studierende/Vorlesung: einwertiger Schlüssel ⇒ BCNF. Kapitel(Modulnr,Überschrift,Inhalt): Inhalt hängt von beiden Teilen ab ⇒ BCNF. hört: nur Schlüsselattribute ⇒ trivial BCNF. Raumbellegung(Raum,Tag,Zeit,Semester,Modulnr) hängt vom vollständigen Schlüssel ab ⇒ BCNF. Prüfung: einwertig ⇒ BCNF. schreibt(Modulnr,PrüfungsID,Matrikelnr,Note): Note hängt von voller Kombination ab ⇒ BCNF. Ergebnis: gesamtes Schema in BCNF.

3.6 Normalform HR-Schema

► **Lösung:** Alle 9 Relationen: einwertiger PK oder zusammengesetzter PK mit voller Abhängigkeit (job_history: Job/Abteilung hängen von Mitarbeiter+Startdatum; employee2project: assoc_pct/is_project_lead hängen von Mitarbeiter+Projekt) ⇒ BCNF. Sonderfälle: (1) postal_code→city wäre transitiv/3NF-Verstoß, gilt hier aber nicht global (NULL-PLZ bei London/Rom etc.) ⇒ kein Verstoß. (2) job.min/max_salary sind aus employee.salary abgeleitet – kein BCNF-Verstoß innerhalb job, aber relationenübergreifende Redundanz (manuell konsistent zu halten).

Aufgabe 4 SQL Data Definition Language – Teil 1

Ziel: Im Rahmen dieser Übung sollen die grundlegenden SQL-Befehle für die Implementierung des HR-Schemas mit Hilfe eines MariaDB-DBS erstellt werden.

4.2 Schema-Erstellung

Neues Schema "hr" erstellen (Datei HR_setupDB.sql, beliebig oft ausführbar).

► **Lösung:** DROP DATABASE IF EXISTS hr; CREATE DATABASE hr; USE hr;

Das DROP DATABASE IF EXISTS stellt sicher, dass das Skript beliebig oft ausgeführt werden kann und das Schema bei jedem Lauf komplett neu aufgebaut wird.

4.3 Relationen, Attribute und Datentypen

Alle Relationen mit Attributen/Datentypen/PK im "hr"-Schema erstellen.

► **Lösung:** CREATE TABLE region (region_id INT NOT NULL, region_name VARCHAR(25), PRIMARY KEY (region_id));

CREATE TABLE country (country_id CHAR(2) NOT NULL, country_name VARCHAR(40), region_id INT NOT NULL, PRIMARY KEY (country_id));

CREATE TABLE location (location_id INT NOT NULL, AUTO_INCREMENT, street_address VARCHAR(40), postal_code VARCHAR(12), city VARCHAR(30) NOT NULL, state_province VARCHAR(25), country_id CHAR(2) NOT NULL, PRIMARY KEY (location_id));

CREATE TABLE department (department_id INT NOT NULL, department_name VARCHAR(30) NOT NULL, location_id INT, PRIMARY KEY (department_id));

CREATE TABLE job (job_id VARCHAR(10) NOT NULL, job_title VARCHAR(35) NOT NULL, min_salary DECIMAL(8,0), max_salary DECIMAL(8,0), PRIMARY KEY (job_id));

CREATE TABLE employee (employee_id INT NOT NULL, first_name VARCHAR(20), last_name VARCHAR(25) NOT NULL, email VARCHAR(25) NOT NULL, phone_number VARCHAR(20), hire_date DATE NOT NULL, job_id VARCHAR(10) NOT NULL, salary DECIMAL(8,2) NOT NULL, commission_pct DECIMAL(2,2), manager_id INT, department_id INT, PRIMARY KEY (employee_id));

CREATE TABLE job_history (employee_id INT NOT NULL, start_date DATE NOT NULL, end_date DATE NOT NULL, job_id VARCHAR(10) NOT NULL, department_id INT NOT NULL, PRIMARY KEY (employee_id, start_date));

CREATE TABLE project (

```
project_id INT NOT NULL,
project_name VARCHAR(50),
PRIMARY KEY (project_id) );

CREATE TABLE employee2project (
employee_id INT NOT NULL,
project_id INT NOT NULL,
assoc_pct DECIMAL(3,2),
is_project_lead BOOL,
PRIMARY KEY (employee_id, project_id) );
```

Optionalität der Attribute: Attribute, die laut Aufgabenstellung immer vorhanden sein müssen (z.B. Namen, Datumswerte, E-Mail), sind NOT NULL; optionale Attribute (z.B. commission_pct, manager_id beim CEO, department_id vor Zuordnung) bleiben nullable.

4.4 Fremdschlüsselbeziehungen

4.4.1 Implementierung

Ergänzen Sie die notwendigen SQL-Befehle, um die Fremdschlüsselbeziehungen zwischen den Relationen unter Berücksichtigung der vorgegebenen Referenzierungsoptionen herzustellen.

► **Lösung:**

```
ALTER TABLE country
ADD FOREIGN KEY (region_id) REFERENCES
region(region_id)
ON UPDATE CASCADE ON DELETE
RESTRICT;
```

```
ALTER TABLE location
ADD FOREIGN KEY (country_id)
REFERENCES country(country_id)
ON UPDATE CASCADE ON DELETE
RESTRICT;
```

```
ALTER TABLE department
ADD FOREIGN KEY (location_id)
REFERENCES location(location_id)
ON UPDATE CASCADE ON DELETE
RESTRICT;
```

```
ALTER TABLE employee
ADD FOREIGN KEY (job_id) REFERENCES
job(job_id)
ON UPDATE CASCADE ON DELETE
RESTRICT,
ADD FOREIGN KEY (department_id)
REFERENCES department(
department_id)
ON UPDATE CASCADE ON DELETE
RESTRICT,
ADD FOREIGN KEY (manager_id)
REFERENCES employee(employee_id)
ON UPDATE CASCADE ON DELETE
RESTRICT;
```

```
ALTER TABLE job_history
ADD FOREIGN KEY (employee_id)
REFERENCES employee(employee_id)
ON UPDATE CASCADE ON DELETE
CASCADE,
ADD FOREIGN KEY (job_id) REFERENCES
job(job_id)
ON UPDATE CASCADE ON DELETE
CASCADE,
ADD FOREIGN KEY (department_id)
REFERENCES department(
department_id)
ON UPDATE CASCADE ON DELETE
CASCADE;
```

```
ALTER TABLE employee2project
ADD FOREIGN KEY (employee_id)
REFERENCES employee(employee_id)
ON UPDATE CASCADE ON DELETE
CASCADE,
ADD FOREIGN KEY (project_id)
REFERENCES project(project_id)
ON UPDATE CASCADE ON DELETE
CASCADE;
```

4.4.2 Verständnisfragen

Erläutern Sie die Auswirkungen folgender Änderungen am Datenbestand unter Berücksichtigung der o.g. Referenzierungsoptionen.

► **Lösung:**

- RESTRICT bei employee(department_id): Abteilung mit Mitarbeitern kann nicht gelöscht werden (FK-Fehler), bis alle Mitarbeiter umgezogen sind.
- location_id ändern: ON UPDATE CASCADE ⇒ automatisch in department übernommen, kein Fehler.
- Location mit Departments löschen: abgelehnt (RESTRICT), Departments müssen zuerst weg.
- Mitarbeiter löschen: als Manager (referenziert über manager_id) ⇒ RESTRICT verhindert es; als Nicht-Manager ⇒ erlaubt, job_history/employee2project-Einträge werden per CASCADE mitgelöscht.
- Alle Lösch-Optionen = CASCADE, Region löschen: Kettenreaktion Region→Country→Location→Department→(inkl. rekursiv über manager_id)→job_history/employee2project – könnte Großteil der DB löschen. Deshalb RESTRICT für die strukturelle Hierarchie, nur CASCADE für Historien-/Detailtabellen.

Aufgabe 5 Constraints

Im Rahmen dieser Aufgabe soll das HR-Schema um weitere Regeln zur automatischen Sicherstellung der Datenintegrität erweitert werden (Ergänzungen in HR_setupDB.sql).

5.1 Arithmetische Constraints

- min_salary in der job-Relation soll positiv sein
- max_salary in der job-Relation soll mindestens so groß wie min_salary sein
- salary und commission_pct in der employee-Relation sollen positiv sein

- `end_date` in der `job_history`-Relation darf nicht vor dem `start_date` liegen
- `assoc_pct` in der `employee2project`-Relation muss größer als 0 und kleiner gleich 1 sein

5.2 Reguläre Ausdrücke

- `street_address` in der `location`-Relation soll an einer beliebigen Stelle mindestens eine Ziffer enthalten (Hausnummer)
- die `email`-Adresse in der `employee`-Relation soll dem Muster `[a-zA-Z][a-zA-Z0-9]*@[a-zA-Z]+\.[a-z]{2,4}` entsprechen

► Lösung (5.1 & 5.2):

```
ALTER TABLE job
ADD CONSTRAINT MinSalaryPositive CHECK
    (min_salary > 0),
ADD CONSTRAINT
    MaxSalaryGreaterThanMinSalary
CHECK (max_salary >= min_salary)
;

ALTER TABLE employee
ADD CONSTRAINT SalaryPositive CHECK (
    salary > 0),
ADD CONSTRAINT CommissionPctPositive
CHECK (commission_pct > 0),
ADD CONSTRAINT EmailPattern
CHECK (email RLIKE '^[a-zA-Z][a-zA-Z0-9]*@[a-zA-Z]+\.[a-z]{2,4}$');

ALTER TABLE job_history
ADD CONSTRAINT EndDateAfterStartDate
CHECK (end_date > start_date);

ALTER TABLE employee2project
ADD CONSTRAINT AssocPctRange CHECK (
    assoc_pct > 0 AND assoc_pct <= 1);

ALTER TABLE location
ADD CONSTRAINT
    AtLeastOneDigitInStreetAddress
CHECK (street_address RLIKE '
[0-9]');
```

Hinweis zum regulären Ausdruck der E-Mail-Adresse: `^` und `$` verankern den Ausdruck an Anfang/Ende; `[a-zA-Z]` erzwingt einen Buchstaben als erstes Zeichen; `[a-zA-Z0-9]*` erlaubt beliebig viele weitere alphanumerische Zeichen vor dem `@`; `[a-zA-Z]+\.` verlangt mindestens einen Buchstaben gefolgt von einem Punkt; `[a-z]{2,4}` erzwingt die 2–4-stellige Top-Level-Domain in Kleinbuchstaben.

5.3 Komplexe Constraints

Identifizieren Sie weitere sinnvolle Constraints, die über die einfache Prüfung der Attributwerte eines neuen Tupels hinausgehen und z.B. bereits vorhandene Tupel mit einbeziehen.

► **Lösung:** CHECK prüft nur die eigene Zeile; für zeilen-/tabellenübergreifende Regeln braucht es TRIGGER. Kandidaten:

1. Mitarbeiter ≠ eigener Vorgesetzter – einfacher CHECK: `ADD CONSTRAINT NotOwnManager CHECK (employee_id <> manager_id)`
2. Gehalt muss in `[min_salary,max_salary]` des Jobs liegen (TRIGGER, Blick in job):

```
DELIMITER $$
CREATE TRIGGER trg_salary_in_range
BEFORE INSERT ON employee
FOR EACH ROW BEGIN
    DECLARE v_min,v_max DECIMAL(8,0);
    SELECT min_salary,max_salary INTO
        v_min,v_max FROM job WHERE
        job_id=NEW.job_id;
    IF NEW.salary NOT BETWEEN v_min AND
        v_max THEN
        SIGNAL SQLSTATE '45000' SET
        MESSAGE_TEXT='Gehalt
        ausserhalb Rahmen';
    END IF;
END$$ DELIMITER ;
```
3. Summe `assoc_pct` je Mitarbeiter ≤ 1 (Aggregation über vorhandene Zeilen; in Aufgabe 8.4.1 bewusst nicht durchgesetzt)
4. `job_history`-Zeiträume je Mitarbeiter dürfen sich nicht überlappen; `start_date` ≥ `hire_date` – analog per TRIGGER

Aufgabe 6 Data Manipulation Language

6.1 Einfügen von Daten

Einfügen: Regionen, fehlende Länder (Italien, Japan, USA, Kanada, Ägypten), Projektzuordnungen: Dellinger→Lean Management (70%), Sully→US Sales Campaign (100%, Leitung), Weiss→Europe Sales Campaign (20%).

► **Lösung:** Die bereitgestellte Datei enthält bereits Standorte, Abteilungen, Jobs, Mitarbeiter, Beschäftigungshistorien und Projekte – es fehlen lediglich die Regionen, ein Teil der Länder sowie die drei neuen Projektzuordnungen:

```
-- Regionen
INSERT INTO region VALUES (1, 'Europe');
INSERT INTO region VALUES (2, 'Americas');
INSERT INTO region VALUES (3, 'Asia');
INSERT INTO region VALUES (4, 'Middle East and Africa');

-- fehlende Laender
INSERT INTO country VALUES ('IT', 'Italy', 1);
INSERT INTO country VALUES ('JP', 'Japan', 3);
INSERT INTO country VALUES ('US', 'United States of America', 2);
INSERT INTO country VALUES ('CA', 'Canada', 2);
INSERT INTO country VALUES ('EG', 'Egypt', 4);

-- Projektzuordnungen
-- Julia Dellinger (employee_id 186) -> Lean Management (project_id 102)
INSERT INTO employee2project VALUES (186, 102, 0.70, FALSE);
```

```
-- Patrick Sully (employee_id 157) -> US Sales Campaign (project_id 100), Projektleiter
INSERT INTO employee2project VALUES (157, 100, 1.00, TRUE);
-- Matthew Weiss (employee_id 120) -> Europe Sales Campaign (project_id 101)
INSERT INTO employee2project VALUES (120, 101, 0.20, FALSE);
```

6.2 Aktualisieren von Daten

- Die neue Telefonnummer von Michael Rogers lautet 650.127.3418
- Der Mindest-Verdienst aller Jobs soll bei Bedarf auf 4000 angehoben werden
- Alle Gehälter sollen jeweils um 3% angehoben werden

► Lösung:

```
UPDATE employee
SET phone_number = '650.127.3418'
WHERE first_name = 'Michael' AND
    last_name = 'Rogers';

UPDATE job
SET min_salary = 4000
WHERE min_salary < 4000;

UPDATE employee
SET salary = salary * 1.03;
```

6.3 Löschen von Daten

- Alle Mitarbeiter, die zu 25% oder weniger zu einem Projekt zugeordnet sind, sollen aus dem jeweiligen Projekt entfernt werden
- Der Mitarbeiter Jonathon Taylor verlässt das Unternehmen. Vergleichen Sie den Inhalt der Relation `job_history` vor und nach der Ausführung des DELETE-Statements.

► Lösung:

```
DELETE FROM employee2project WHERE
    assoc_pct <= 0.25;

-- vorher: Inhalt der Job_History von Jonathon Taylor (employee_id 176) prüfen
SELECT * FROM job_history WHERE
    employee_id = 176;
-- liefert 2 Zeilen: SA_REPL (1998-03-24 bis 1998-12-31) und SA_MAN (1999-01-01 bis 1999-12-31)

DELETE FROM employee WHERE first_name = 'Jonathon' AND last_name = 'Taylor';

-- nachher:
SELECT * FROM job_history WHERE
    employee_id = 176;
-- liefert 0 Zeilen
```

Da `job_history(employee_id)` mit `ON DELETE CASCADE` auf `employee(employee_id)` verweist, werden beim Löschen des Mitarbeiters automatisch *beide* zugehörigen Beschäftigungshistorien-Einträge mitgelöscht – ohne dass ein explizites DELETE auf `job_history` nötig wäre.

6.4 Testen der Constraints

Testen Sie die Check-Constraints aus Aufgabe 5, indem Sie jeweils ein INSERT-Statement für einen gültigen Datensatz und einen Datensatz, der das jeweilige Constraint verletzt, erstellen und ausführen.

► **Lösung:** Testprinzip: je 1 INSERT mit gültigem und 1 mit verletzendem Wert, Rest der Spalten beliebig gültig gefüllt.

- `MinSalaryPositive`: `min_salary=3000` OK / –100 Fehler
- `MaxSalaryGreaterThanMinSalary`: (3000,6000) OK / (5000,3000) Fehler
- `SalaryPositive`: `salary=4000` OK / –500 Fehler
- `EndDateAfterStartDate`: (2000-01-01,2001-01-01) OK / (2001-01-01,2000-01-01) Fehler
- `AssocPctRange`: `assoc_pct=0.5` OK / 1.5 Fehler
- `AtLeastOneDigitInStreetAddress`: 'Musterstr. 12' OK / 'Musterstrasse ohne Nr' Fehler
- `EmailPattern`: 'tmail@example.com' OK / 'ungueltige-mail-adresse' Fehler

Aufgabe 7 SELECT Statement – Grundlagen

Im Rahmen dieser Übung sollen Informationen aus dem `hr`-Schema ausgegeben werden.

7.1 einfaches SELECT-Statement

Namen aller Länder ausgeben.

► Lösung:

```
SELECT country_name FROM hr.country;
```

7.2 aktuelle Uhrzeit

Aktuelle Uhrzeit als "HH:MM" ausgeben.

► Lösung:

```
SELECT NOW(), DATE_FORMAT(NOW(), '%H:%i')
FROM DUAL;
```

7.3 DISTINCT

Duplikatfrei alle Manager-IDs liefern.

► Lösung:

```
SELECT DISTINCT manager_id FROM hr.
employee WHERE manager_id IS NOT
NULL;
```

7.4 String-Operationen

Adressinfos aller Locations; Spalte "Stadt" im Format <country_id> - <postal_code> <city> <state_province>, NULL→"unbekannt".

► Lösung:

```
SELECT
    street_address AS Adresse,
    IFNULL(
```

```
CONCAT(country_id, '-',
    postal_code, ', ', city, ' (',
    state_province, ')'),
'unbekannt'
) AS Stadt
FROM hr.location;
```

7.5 Filtern

Alle Infos des CEO (= Mitarbeiter ohne Vorgesetzten) ausgeben.

► Lösung:

```
SELECT *
FROM hr.employee
WHERE manager_id IS NULL;
```

7.6 Filtern nach mehreren Kriterien

job: Jobs mit Mindestgehalt > 2000 und Bezeichnung enthält "Clerk".

► Lösung:

```
SELECT *
FROM hr.job
WHERE min_salary > 2000 AND LOWER(job_id)
    LIKE '%clerk%';
```

7.7 Subqueries

Alle Mitarbeiter mit Managerposition (vollständig).

► Lösung:

```
SELECT *
FROM hr.employee
WHERE employee_id IN (
    SELECT DISTINCT manager_id FROM hr.
    employee WHERE manager_id IS NOT
    NULL
);
```

7.8 komplexes SQL-Statement

Name, E-Mail, Telefon, Einstellungsdatum (←Betriebszugehörigkeit in Jahren zum 01.01.2023, 1 Nachkommastelle), Gehalt (kEUR), Vorgesetzter (bzw. "niemand") aller Manager mit Gehalt >10000, eingestellt vor 01.01.1996.

► **Lösung:** Das Ergebnis muss **exakt** der vorgegebenen Formatierung entsprechen (Groß-/Kleinschreibung, Abkürzungen, Attributzusammenfassung, NULL-Behandlung):

```
SELECT
    CONCAT(first_name, ' ', last_name) AS
        Name,
    REPLACE(LOWER(email), '@example.com',
        '@...') AS Email,
    SUBSTR(phone_number, 5) AS Telefon,
    CONCAT(
        DATE_FORMAT(hire_date, '%d.%m.%Y')
        , ' ',
        ROUND(DATEDIFF(STR_TO_DATE('
01.01.2023','%d.%m.%Y'),
            hire_date) / 365, 1),
        ' Jahre')
    ) AS Einstellungsdatum,
    CONCAT(ROUND(salary/1000, 0), ' k EUR')
    AS Gehalt,
    IFNULL(manager_id, 'niemand') AS
        Vorgesetzter
FROM hr.employee
WHERE
    salary > 10000 AND
    hire_date < STR_TO_DATE('1996-01-01',
        '%Y-%m-%d') AND
    employee_id IN (SELECT DISTINCT
        manager_id FROM hr.employee
        WHERE manager_id IS NOT NULL);
```

Aufgabe 8 SELECT Statement – Teil 2

8.1 Sortieren von Tupeln

8.1.1 Telefonbuch

Alphabetisch aufsteigend nach Nach- und Vornamen sortierte Liste (Nachname, Vorname, Telefonnummer) aller Mitarbeiter.

► Lösung:

```
SELECT last_name, first_name, phone_number
FROM hr.employee
ORDER BY last_name ASC, first_name ASC;
```

8.1.2 Telefonbuch zur Rückwärtssuche

Aufsteigend nach Telefonnummer sortierte Liste (Telefonnummer, Nachname, Vorname).

► Lösung:

```
SELECT phone_number, last_name, first_name
FROM hr.employee
ORDER BY phone_number ASC;
```

8.2 Gruppieren von Tupeln

8.2.1 Anzahl neuer Mitarbeiter nach Einstellungsjahr

► Lösung:

```
SELECT
    DATE_FORMAT(hire_date, '%Y') AS
        Einstellungsjahr,
    COUNT(employee_id) AS Mitarbeiterzahl
FROM hr.employee
GROUP BY Einstellungsjahr;
```

8.2.2 Anzahl der Standorte pro Land

Absteigend nach Anzahl der Standorte sortiert.

► Lösung:

```
SELECT
    country_id AS Laendercode,
    COUNT(location_id) AS Standortanzahl
FROM hr.location
GROUP BY country_id
ORDER BY Standortanzahl DESC;
```

8.3 Verknüpfung von Relationen

8.3.1 Abteilungsdetails

Liste aller Abteilungen mit Region, Land, Bundesland, Adresse (PLZ, Ort, Anschrift) und Abteilungsname.

► Lösung:

```
SELECT
    region.region_name,
```

```
country.country_name,
location.state_province,
CONCAT(location.postal_code, ', ',
    location.city, ', ', location.
    street_address) AS Adresse,
department.department_name
FROM hr.department
JOIN hr.location ON department.
    location_id = location.
    location_id
JOIN hr.country ON location.country_id
    = country.country_id
JOIN hr.region ON country.region_id =
    region.region_id;
```

8.3.2 Zuordnung von Mitarbeitern zu Projekten

Liste mit Projektname, Vor-/Nachname des Mitarbeiters und Anteil der Arbeitszeit.

► Lösung:

```
SELECT
    project.project_name,
    employee.first_name,
    employee.last_name,
    employee2project.assoc_pct
FROM hr.project
INNER JOIN hr.employee2project ON
    project.project_id =
    employee2project.project_id
INNER JOIN hr.employee ON employee.
    employee_id = employee2project.
    employee_id;
```

8.4 Gruppieren von Tupeln aus verknüpften Tabellen

8.4.1 Überlastete Mitarbeiter

Liste der Mitarbeiter, die zu mehr als 100% zu Projekten zugeordnet sind (Vor-/Nachname, Gesamtauslastung, Komma-separierte Projektliste).

► Lösung:

```
SELECT
    employee.last_name AS Nachname,
    employee.first_name AS Vorname,
    SUM(employee2project.assoc_pct) AS
        Gesamtauslastung,
    GROUP_CONCAT(project.project_name
        SEPARATOR ', ') AS Projekte
FROM hr.project
INNER JOIN hr.employee2project ON
    project.project_id =
    employee2project.project_id
INNER JOIN hr.employee ON employee.
    employee_id = employee2project.
    employee_id
GROUP BY employee.first_name, employee.
    last_name
HAVING Gesamtauslastung > 1;
```

Diese Abfrage liefert bei den Beispieldaten aus Aufgabe 6.1 genau die Zeile Baer | Hermann | 1.10 | Europe Sales Campaign, US Sales Campaign – da die Constraint aus Aufgabe 5.3 (Summe der Auslastung ≤ 100%) bewusst *nicht* als harter CHECK umgesetzt wurde, konnte dieser Datensatz überhaupt erst entstehen.

Aufgabe 9 Mengenoperationen und Views

9.1 Mengenoperationen

Bonus für: 3 Mitarbeiter mit niedrigstem Gehalt unter den bis 1996 Eingestellten sowie 3 Mitarbeiter mit insgesamt niedrigstem Gehalt. Personenkreis nach Gehalt aufsteigend sortiert ermitteln.

► Lösung:

```
(SELECT employee_id, first_name, last_name
    , hire_date, job_id, salary
FROM hr.employee
WHERE DATE_FORMAT(hire_date, '%Y') <=
    1996
ORDER BY salary ASC LIMIT 3)
UNION
(SELECT employee_id, first_name, last_name
    , hire_date, job_id, salary
FROM hr.employee
ORDER BY salary ASC LIMIT 3)
ORDER BY salary ASC;
```

UNION entfernt automatisch Duplikate (z.B. Mitarbeiter, die in beiden Teilmengen vorkommen), sodass am Ende höchstens 6, ggf. aber auch weniger Zeilen ausgegeben werden.

9.2 Views

View `employee_details` mit `employee_id`, `job_id`, `manager_id`, `department_id`, `location_id`, `country_id`, `first/last_name`, `salary`, `commission_pct`, `department_name`, `job_title`, `city`, `state_province`, `country_name`, `region_name`. Danach möglichst einfach abfragen.

► Lösung:

```
CREATE OR REPLACE VIEW employee_details AS
SELECT
```

```
    employee_id, job_id, manager_id,
    department_id, location_id,
    country_id,
    first_name, last_name, salary,
    commission_pct,
    department_name, job_title, city,
    state_province, country_name,
    region_name
FROM hr.employee
JOIN hr.department USING (
    department_id)
JOIN hr.job USING (job_id)
JOIN hr.location USING (location_id)
JOIN hr.country USING (country_id)
JOIN hr.region USING (region_id);
```

-- lesender Zugriff:
SELECT * FROM employee_details;
Die Verwendung von USING(...) ist hier möglich, weil die verknüpfenden Fremdschlüsselspalten in beiden beteiligten Tabellen jeweils identisch benannt sind (department_id, job_id, location_id, country_id, region_id); USING vermeidet dabei, dass die gleichnamige Spalte doppelt im Ergebnis erscheint.